taskCenture

seu app de gerenciamento de tarefas

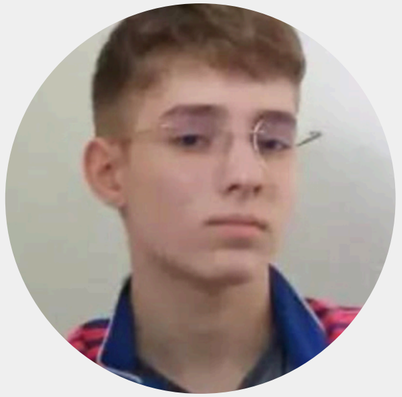
Grupo 8:

Kamila Queiroz, Larry Diego e Leonardo Ramalho

</nosso-time>

Kamila Queiroz

Larry Diego



Leonardo Ramalho

</desafio-proposto>

A Accenture precisava de um sistema de gerenciamento de tarefas:

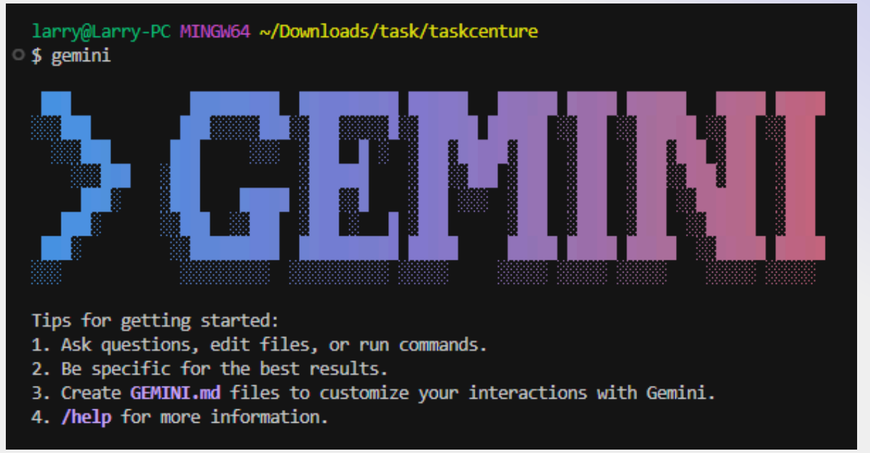
- **01** Gestão de usuários e seus cargos (autenticação e autorização);
- **02** Gerenciar projetos e atividades;
- **03** Registrar comentários e histórico de alterações das tarefas;
- **04** Garantir organização e colaboração de forma eficiente.

</nossa-solução>

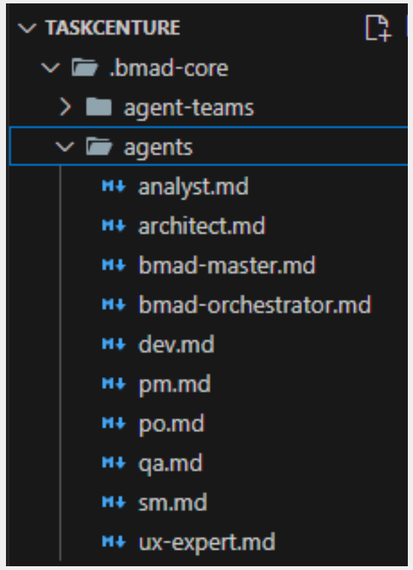
Uma aplicação web completa: taskCenture

- **01** Full Stack (Backend + Frontend);
- **02** Divisão de responsabilidades por cargos;
- **03** Gestão de projetos e tarefas com fluxo Kanban;
- **04** Utilização do método BMAD para auxiliar no desenvolvimento.

É uma metodologia que utiliza agentes de IA especializados para simular papéis reais dentro de um ciclo de desenvolvimento ágil.



Cada agente representa um papel do time no ciclo de desenvolvimento:

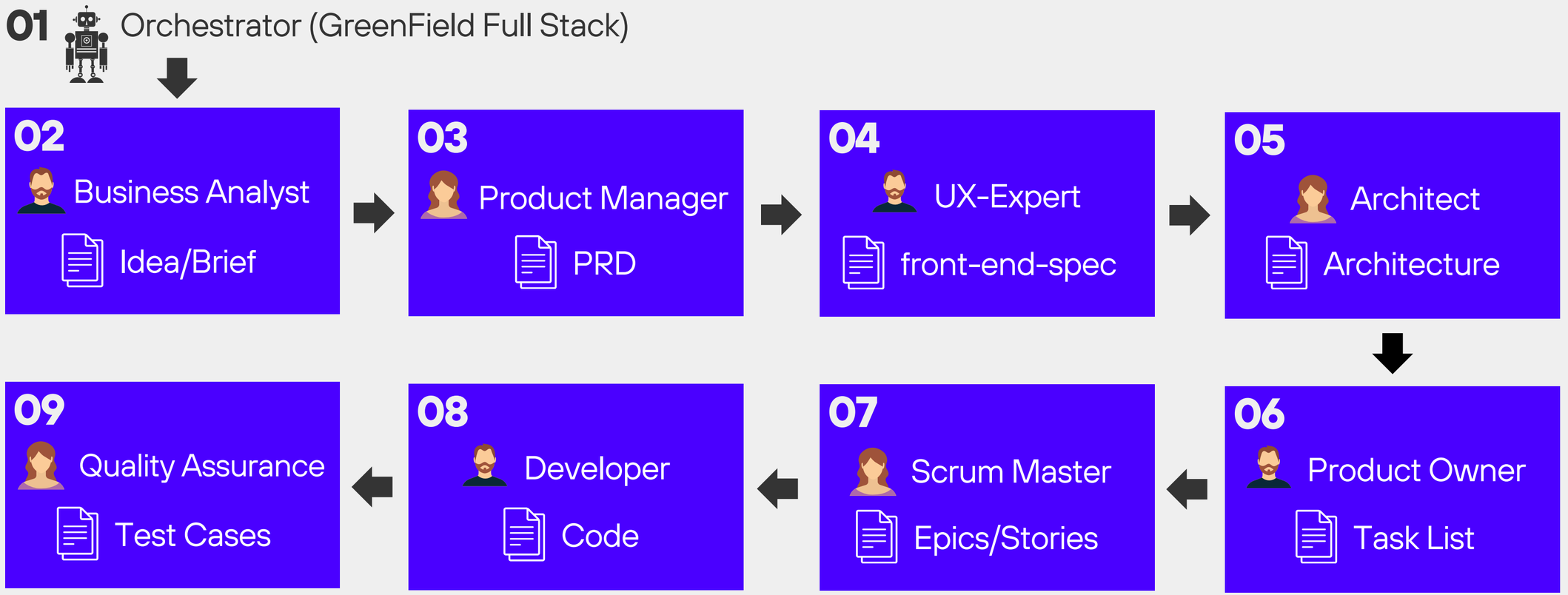


</método-bmad>

</fluxo-bmad>

Cada agente gera um documento específico após uma série de perguntas;

Iniciamos o projeto conversando com o “Orchestrator”, que direcionava cada etapa:



</conversa>

Greetings! I am Winston, your Architect. My role is to design robust, scalable, and maintainable systems. I will now define the technical architecture for your project, ensuring it aligns with the PRD and supports the UX design we just discussed.

Based on the PRD's "Technical Assumptions" and the overall project goals, I will focus on creating a comprehensive architecture document.

Conversa com o Arquiteto

✦ I have successfully created the architecture document at docs/architecture.md.

Now, as per our plan, the next step is to validate this architecture against the architect-checklist.md to ensure it is robust, complete, and aligned with our goals.

Would you like to proceed with the checklist validation?

1. **Yes, let's run the checklist.**
2. **No, I want to review the document first.**

Validação do documento architecture.md

✦ Understood. Let's now define the **Security and Error Handling Strategy**.

Ensuring the system is secure and handles errors gracefully is paramount for reliability and user trust.

Security Strategy

Our security strategy will be built upon the principles of least privilege, defense-in-depth, and secure-by-design.

1. Authentication & Authorization

* **Authentication Method:** JWT-based authentication using Spring Security.
  * Users log in with credentials, receive a JWT.
  * JWT is sent with every subsequent request to protected resources.
* **Authorization Model:** Role-Based Access Control (RBAC) enforced at the API layer using Spring Security.
  * Roles (ADMINISTRATOR, MANAGER, COLLABORATOR) will be associated with users.
  * API endpoints will be secured based on required roles (e.g.,

Definição dos requisitos de segurança com o Product Manager

</conversa>

> Just being sure, the ROLE Collaborator should only be able to complete a task, be assigned to a task, create/delete/edit a comment, and be a part of a project. Managers should be able to create a project, edit, and delete it, and also assign tasks to a collaborator, and, obviously, have all the permissions of the Collaborator role. And administrator should have all the permissions mentioned above, but also be able to manage users, create, delete, edit them, etc.

✦ That is an excellent and crucial clarification. Your understanding is perfectly correct and aligns with the intent of the Role-Based Access Control (RBAC) defined in the PRD.

Let me formalize that hierarchy and set of permissions for absolute clarity. This will be the guiding principle for implementing security rules in the backend.

Divisão de responsabilidades por cargos

Role-Based Access Control (RBAC) Breakdown:

1. ROLE_COLLABORATOR (Base Permissions)
A COLLABORATOR is focused on executing work within projects they are part of.
* **Tasks:**
    * Can be assigned to tasks.
    * Can update the status of their *own* assigned tasks (e.g., move from 'IN_PROGRESS' to 'DONE').
    * Can view all tasks within projects they are a member of.
* **Comments:**
    * Can create comments on tasks within their projects.
    * Can edit or delete their *own* comments.
* **Projects:**
    * Can view projects they are a member of.
* **Users:**
    * No permissions.

1. Role - Collaborator

2. ROLE_MANAGER (Collaborator+ Project Management)
A MANAGER has all the permissions of a COLLABORATOR, plus the ability to manage the projects they own.
* **Inherits all `ROLE_COLLABORATOR` permissions.**
* **Projects:**
    * Can create new projects (and become the owner).
    * Can edit the details (name, description) of projects they own.
    * Can delete projects they own.
    * Can add/remove users from projects they own.
* **Tasks:**
    * Can create new tasks within projects they own.
    * Can assign or re-assign tasks to any user within the project.
    * Can edit or delete any task within projects they own.

2. Role - Manager

3. ROLE_ADMINISTRATOR (Manager + System Administration)
An ADMINISTRATOR has full system-wide permissions.
* **Inherits all `ROLE_MANAGER` and `ROLE_COLLABORATOR` permissions.**
* **User Management:**
    * Can create new users.
    * Can edit any user's profile and roles.
    * Can delete users.
    * Can view all users in the system.
* **Project Management (Global):**
    * Can view, edit, and delete *any* project in the system, regardless of ownership.

This detailed breakdown ensures that the security logic, which will be built upon the data models we've defined, will be implemented exactly as you've described. Thank you for ensuring we are aligned on this critical detail.

Does this formal breakdown match your expectations perfectly?

3. Role - Administrator

</conversa>

Greetings! I am the UX Expert. My focus is on ensuring the application is intuitive, accessible, and visually appealing. I will now analyze the PRD to create a design concept that aligns with the product's goals.

Based on the PRD, my first step will be to flesh out the core screens and user flows. I will start by creating a more detailed specification for the main views:

1. **Login / Register Screen**
2. **Dashboard Screen**
3. **Project View (Kanban Board)**
4. **Task Detail Modal**

I will begin by designing the **Login and Registration** flow. I will then present the design for your review before moving to the next screen. Does that sound like a good plan?

Conversa com o UX Expert

"Sign In" Tab (Default)

* **Header:** "Welcome Back!"
* **Form Fields:**
    1. **Email Address:** (Input field, type="email", required)
    2. **Password:** (Input field, type="password", required)
* **Actions:**
    * A primary button labeled **"Sign In"**. This button will be disabled until both fields are filled.
    * A subtle link below the button: "Forgot Password?" (This will be out of scope for the MVP as per the PRD, but we'll place it for future implementation).
* **Error Handling:**
    * If login fails (invalid credentials), a single, non-specific error message will appear (e.g., "Invalid email or password.") to avoid user enumeration attacks.

Definição da composição da tela de Login

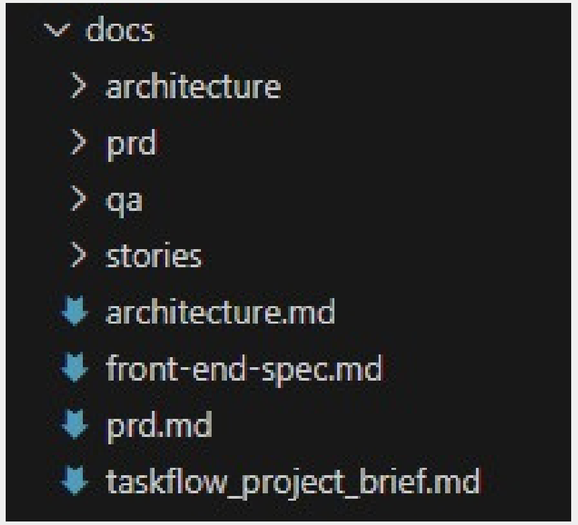
✦ Here is the detailed breakdown for **Epic 3: Kanban Board & Basic Collaboration**:

Expanded Goal:
"The objective of Epic 3 is to enhance task visualization and facilitate basic team collaboration through an interactive Kanban board and essential communication features. This epic will enable users to intuitively manage task statuses via drag-and-drop, add comments to tasks for discussion, and receive basic notifications for important updates. By completing this epic, the system will provide a more dynamic and communicative environment for teams to work together on projects."

Stories for Epic 3:

* **Story 3.1: Kanban Board Display**
    * As a user, I want to view tasks organized in a Kanban board with "To Do," "In Progress," and "Done" columns, so that I can easily visualize the status of tasks within a project.
    * **Acceptance Criteria:**
        1. User can access a Kanban board view for a selected project.
        2. Tasks are displayed as cards within their respective status columns.
        3. Each task card shows key information (e.g., title, assignee, due date).

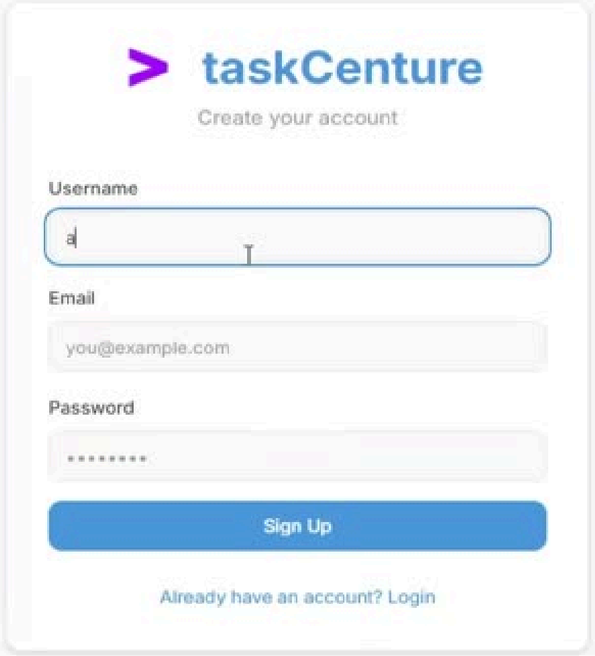
Criação das histórias pelo Scrum Master



Documentos finais

</dificuldades>

- Incompatibilidade de versões ao final do fluxo:
  - JWT (versão antiga incompatível com a versão do Java);
  - Radix UI × versão do React.
- Erros de integração entre backend e frontend;
- Necessidade de ajustes manuais no código gerado;
- Apesar disso, o método guiou bem o desenvolvimento, agilizando o entendimento e documentação do sistema.


taskCenture
Create your account
Username
a
Email
you@example.com
Password
Sign Up
Already have an account? Login

Backend

- Java Spring Boot;
- MySQL;
- JWT;
- JUnit (Test).

Frontend

- React + Vite;
- Tailwind CSS;
- Radix UI;
- TypeScript;
- Jest (Test).

Controle e documentação

- BMAD;
- Git + GitHub.

</stack>

</conclusão>

- O BMAD Method mostrou-se uma abordagem inovadora para guiar o desenvolvimento de software;
- Apesar dos desafios técnicos, a aplicação final atingiu o objetivo de gerenciar projetos e tarefas de forma colaborativa e segura;
- O uso de IA no apoio ao ciclo de desenvolvimento representa um passo importante rumo à automação inteligente de processos ágeis.

Obrigado!